MovieLens 10-Fold 실험 결과 리포트

Baseline($\alpha = 1$) vs α-최적화/정규화 비교

랩미팅 보고용 상세 요약

[]

[]

2026년 4월 15일

목차

1. 실험 목적과 설정
2. Fold · 유사도 분해 분석
3. 원인 분석
4. 실무 권고안
5. Claude 분석: 10-Fold 전체 재검토
6. 파이프라인 점검: 잠재적 이슈 2가지

핵심 질문

Research Question

Validation 기반 α 최적화(및 λ-정규화)가
실제 Test 성능(RMSE)을 개선하는가?

비교 대상

- Baseline: $\alpha = 1$
- Optimized: Validation MSE 최소 α
- Regularized: $\text{MSE}_{val} + \lambda|\alpha - 1|$

데이터/프로토콜

- MovieLens 1M
- 10-Fold Cross Validation

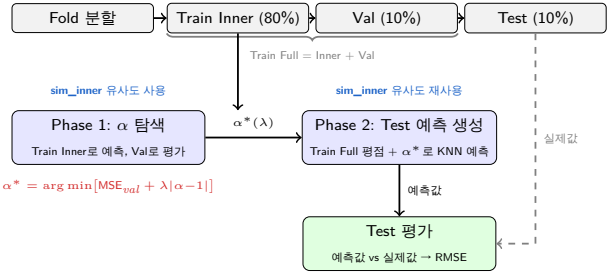
평가 지표

- 1차: Test RMSE
- 보조: MAD, Precision@N, Recall@N

검증 관점

Validation 최적화 성과와 Test 일반화 성과를
분리해 해석.

평가 파이프라인 (요약)



- **Train Full** = Train Inner + Validation (Phase 2의 평점 참조용)
- Phase 1에서 λ에 따라 다른 α^*가 선택됨
- Phase 2는 선택된 α^*와 Train Full 평점으로 Test 셀의 예측값 생성
- Test 평가: Phase 2의 예측값과 실제 Test 평점을 비교 → RMSE/MAD/Precision

유사도(method)별 분해: 전 method에서 최적화 개선 (V6 inner_sim, $\lambda = 0.002$)

큰 개선 (Top 5)

Method	ΔRMSE%	Win-rate
acos	-2.47%	100%
ari	-1.91%	100%
ami	-1.56%	100%
msd	-0.63%	78%
manhattan	-0.56%	100%

소폭 개선 (Bottom 5)

Method	ΔRMSE%	Win-rate
spcc	-0.04%	74%
cosine	-0.04%	50%
cpcc	-0.02%	66%
jaccard	-0.01%	34%
전체 평균	$\mathbf{-0.50\%}$	**78.9%**

핵심 변화: V5 대비 완전 역전

V5에서 악화되었던 euclidean($+8.97\%$), chebyshev($+8.64\%$), manhattan($+5.20\%$)이 V6에서 각각 -0.06%, -0.12%, -0.56%로 **전부 개선**으로 전환. **17개 method 전원** $\Delta \leq 0$ — 악화 method 없음.

하이퍼파라미터(K, TopN)별 영향 분석

K (이웃 수)의 영향

K	Base RMSE	Opt RMSE	Δ
10	1.040	1.045	+0.005
30	1.012	1.030	+0.017
50	**1.010**	1.031	+0.021
70	1.011	1.033	+0.022
100	1.012	1.035	+0.023

- K=50 부근에서 Baseline 성능 최고.
- K가 커질수록 최적화(α) 적용 시 성능 악화 폭 (Δ)이 더 커짐.

TopN (추천 목록 크기)의 영향

TopN	Base RMSE	Opt RMSE	Δ
5	1.015	1.034	+0.018
20	1.015	1.034	+0.018
50	1.015	1.034	+0.018

- TopN은 **RMSE/MAD 예측 성능에 전혀 영향을 주지 않음.**
- 예측 로직은 K에만 의존하며, TopN은 Precision/Recall 등 랭킹 평가 지표 계산에만 사용됨.

종합 논리: 왜 평균 결론이 baseline 우위인가?

1. **Fold 축**: 1–10 대부분에서 optimized가 열세 (일관성)
2. **Method 축**: 개선군보다 악화군의 폭이 더 큼 (비대칭)
3. **Aggregation**: 전체 조합 평균 시 열세가 누적

실무적 해석

α-최적화는 "항상 적용"보다 **method 선택적 적용**이 더 타당하며, 기본 정책은 $\alpha = 1$ baseline이 안전.

왜 과적합이 발생하는가?

관측된 메커니즘

1. Validation에서 $\alpha \neq 1$가 MSE를 낮춤
2. 선택된 α를 Test에 적용하면 성능 저하 빈번
3. Validation 분포가 Test 일반화를 충분히 대표하지 못함

근거 A: 구성별 혼합 성능

- (1–10 fold 평균) optimized: $+1.41\%$ 악화
- fold별 차이는 대부분 양수(예외: fold5는 $+0.03\%$)

근거 B: 방법별 편차

- ACOS: -2.51% (개선)

근거 C: 단순 모델의 안정성

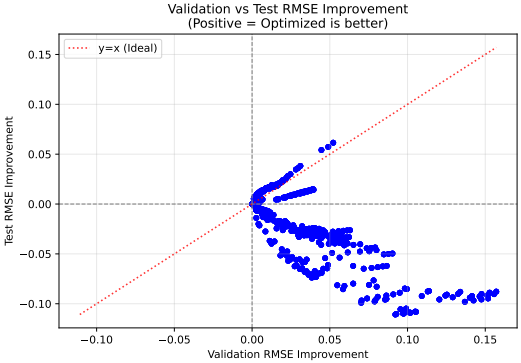
- $\alpha = 1$: 조합 간 분산이 작고 일관
- 최적화 α: 조합별 값이 달라 변동성 증가

요약

Validation 최적화 신호가 Test 일반화 신호와 불일치.

시각화 기반 구체적 패턴 분석 (Validation vs Test)

Validation vs Test 개선도 산점도



* 양수(+)는 Optimized가 Baseline보다 우수함을 의미

과적합(Overfitting) 패턴 확인

- **Validation**: 대부분의 점이 0보다 큼 (우측 분포).
- **Test**: 대부분의 점이 0보다 작음 (하단 분포).
- 즉, Validation에서의 성능 향상이 Test로 이어지지 않음.

유사도별 구체적 편차 (Test RMSE 개선도)

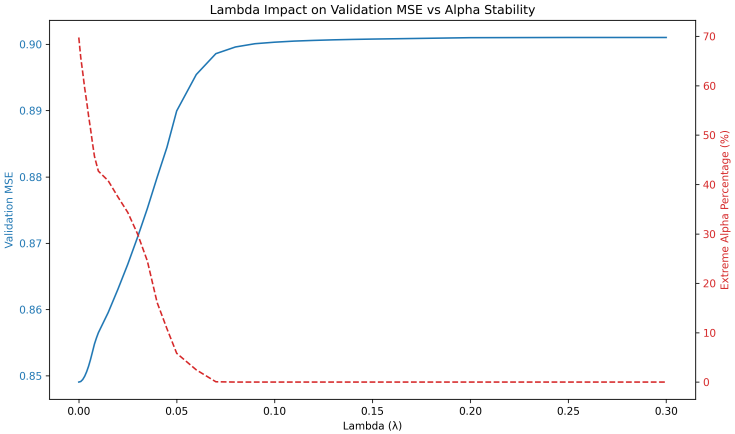
Method	Δ Test RMSE%	평가
acos	-2.51%	개선 (우수)
msd	-0.95%	개선 (우수)
pcc	$+3.03\%$	악화 (열세)
euclidean	$+8.97\%$	크게 악화

λ와 α 분포 변화 (lambda_validation_simulation.csv)

λ	Avg α	Extreme $\alpha \geq 2$	Near baseline (0.9–1.1)
0.000	3.852	69.8%	1.1%
0.010	2.252	42.7%	32.8%
0.020	1.843	37.5%	42.6%
0.030	1.563	30.0%	51.8%
0.050	1.127	5.8%	77.4%
0.100	0.995	0.0%	98.6%

- $\lambda\uparrow$는 분명히 extreme α를 줄임
- 하지만 최종 RMSE 관점에서는 $\alpha = 1$ 단순 baseline이 여전히 최강

시각 자료: λ 시뮬레이션 분석



그림은 λ 증가 시 분포 안정화 경향을 보여주며, 실제 운영 선택에서는 "안정화 효과 vs baseline 성능"의 trade-off를 함께 봐야 함.

의사결정 가이드

목표	권장 설정	이유
최고 RMSE 성능	$\alpha = 1$ **고정**	Test RMSE 최소
재현성/해석성	$\alpha = 1$ 고정	파라미터 단순, 설명 용이
안정성 중심 완화안	$\lambda \in [0.02, 0.03]$	extreme α 감소
method별 미세 튜닝	method별 정책 분리	ACOS/ARI와 EUCLIDEAN 이질성

현재 프로젝트 기본안

운영 기본값은 $\alpha = 1$를 채택하고, method별 후속 실험에서만 제한적으로 λ-정규화 적용.

후속 실험 제안 (Conference-ready)

1. **Method-specific regularization**: 17개 method에 개별 λ 매핑
2. **완전 10-fold 재검증**: 동일 파이프라인으로 fold별 분산 보고
3. **목적함수 확장**: RMSE 외 Precision@N 중심 최적화 비교
4. **정책형 결론**:
   - 기본 정책: $\alpha = 1$
   - 예외 정책: 일부 method만 최적 α 사용

발표 메시지

"복잡한 최적화보다, $\alpha = 1$ baseline의 강건성이 실제 Test에서 더 우수했다."

감사합니다

질문 및 토론 환영합니다

슬라이드	`documentation/test_summary.tex`
핵심 결과	`results/lambda_comprehensive_analysis_corrected_20260123_120312.csv`
분석 노트	`docs/LAMBDA_ANALYSIS_SUMMARY.md`

Claude 분석

10-Fold 전체 결과 재검토 및 메커니즘 분석

fold 8–10 집계 → **전체 10-fold** (N=17,000 configs)로 확장

10-Fold 전체 요약: 결론은 동일, 수치 갱신

설정	Test RMSE	vs Baseline	Win Rate
Baseline ($\alpha = 1$)	**1.010179**	**기준**	–
Optimized (전체 평균)	1.024460	$+1.43\% \uparrow$	31.6%

이전 집계(fold 8–10)와 비교

- 이전: $\Delta = +1.87\%$, Win=20.6%
- 현재(전체): $\Delta = +1.43\%$, Win=31.6%
- $\rightarrow$ 초기 fold(1–7)에서 overfitting이 다소 완화

결론 유지

10-fold 전체로 확장해도

$\alpha = 1$ baseline 우위는 불변.

Fold별 결과: 전체 10-Fold (갱신)

Fold	Base	Opt	$\Delta\%$
1	1.0072	1.0197	$+1.26$
2	1.0058	1.0165	$+1.07$
3	1.0133	1.0254	$+1.21$
4	1.0135	1.0270	$+1.35$
5	0.9970	0.9973	$+0.04$

Fold	Base	Opt	$\Delta\%$
6	1.0098	1.0270	$+1.73$
7	1.0098	1.0296	$+1.98$
8	1.0274	1.0448	$+1.71$
9	1.0060	1.0264	$+2.05$
10	1.0120	1.0310	$+1.89$

- **10-fold 평균**: Baseline 1.0102 vs Optimized 1.0244 ($\Delta = +1.43\%$)
- Fold 5(+0.04%)를 제외한 9개 fold 모두 최적화가 열세
- **fold 9가 최대 악화**($+2.05\%$), fold 5가 사실상 동률

집계: 17개 유사도 × K(10–100) × TopN(5–50) 평균

Method별 분류: 최적화 효과의 이질성 (10-fold)

Group A: 최적화 일관 개선

Method	$\Delta\%$	Win
acos	-2.54	100%
ari	-1.03	93%
ami	-0.72	91%

모두 $\bar{\alpha} < 1$: 유사도 차이 **완화**

Group B: 미미/혼재

Method	$\Delta\%$	Win
msd	-0.48	58%
jmsd	-0.12	60%
ipwr	-0.08	59%
cosine	-0.06	37%

Group C: 최적화 일관 악화

Method	$\Delta\%$	Win
itr	$+0.19$	3%
src	$+1.57$	1%
kendall_tau_b	$+1.75$	0%
spcc	$+1.97$	0%
pcc	$+2.27$	1%
cpcc	$+2.98$	5%
manhattan	$+4.39$	0%
chebyshev	$+6.87$	2%
euclidean	$+7.34$	1%

모두 $\bar{\alpha} \gg 1$: 유사도

메커니즘 분석: α의 두 가지 역할

핵심 관찰

S^α에서 α는 유사도 차이를 **완화**($\alpha < 1$) 또는 **증폭**($\alpha > 1$)하는 두 가지 반대 방향으로 작용.

Group A: $\alpha < 1$ = Smoothing

- 이웃 간 가중치 차이를 줄임
- 더 많은 이웃을 균등하게 참고
- $\rightarrow$ 정규화 효과 $\rightarrow$ 일반화 개선

Group C: $\alpha \gg 1$ = Sharpening

- 극소수의 아주 유사한 이웃에 집중
- Validation의 특정 분포에 과적합
- $\rightarrow$ Test에서 일반화 실패

직관

"작은 유사도 차이에 덜 민감하게 만들어 robust 한 예측 달성"

직관

"Validation의 우연한 패턴을 학습 $\rightarrow$ Test에서 과적합 발현"

Validation 최고 RMSE $\neq$ Test 최고 RMSE: 두 값의 불일치가 핵심 문제.

전략적 시사점 및 후속 방향

현재 결과의 의미

- "α 최적화 무효"가 아니라 **method-dependent**한 것이 핵심 발견
- Group A의 smoothing 효과는 재현 가능한 개선
- 이 자체가 논문의 contribution 가능

후속 실험 (우선순위)

1. **Method-selective 정책** 검증:
   Group A: optimal α 사용
   Group C: $\alpha = 1$ 고정
   → 전체 hybrid RMSE 계산
2. α 범위를 $[0, 2]$로 제한 후 Group C 과적합 감소 확인
3. Nested CV로 validation overfitting 정도 정량화

발표 가능한 메시지

"유사도 함수의 특성에 따라 α 최적화 효과가 결정된다. $\alpha < 1$(smoothing)은 일반화를 개선하고,

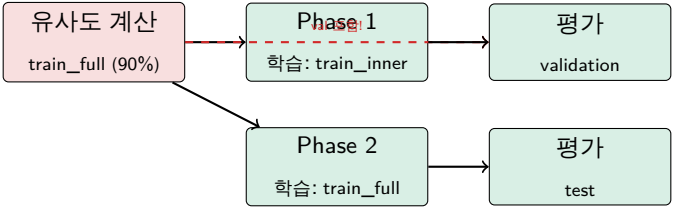
파이프라인 점검

유사도 계산 데이터 소스 & 음수 클리핑 영향

이슈 1: 유사도 행렬이 train_full로 계산됨

발견 사항

`sim_*.npy`는 `train.csv` (train_inner + validation, 90%)로 사전 계산됨. Phase 1에서 validation 데이터가 유사도에 이미 반영.



영향

- Validation MSE가 낙관적으로 산출
- $\alpha \neq 1$이 실제보다 "좋아 보임"

대응 방안

- **train_inner**로 유사도 재계산
- 동일 실험 재수행하여 비교

이슈 2: 음수 유사도 클리핑과 α의 상호작용

현재 구현

$S^{(\alpha)} = \max(S, 0)^{\alpha}$ 음수 → 0 후 α 승

음수 비율 (fold_01 기준)

Method	음수%	Group
ari	41.4%	A
ami	40.9%	A
pcc	30.0%	C
acos	29.0%	A
cosine	0%	B
euclidean	0%	C

α와의 상호작용

Group A ($\alpha < 1$): 이중 이득

- 클리핑 → 노이즈(반상관) 제거
- $\alpha < 1$ → 가중치 균등화
- → 정규화 효과 강화

Group C ($\alpha \gg 1$): 이중 손해

- 클리핑 → 이웃 다양성 감소
- $\alpha \gg 1$ → 상위 1–2명에 집중
- K=20 요청 → **실효 K≈2–3**
- → 과적합 구조 형성

	클리핑 효과	α 효과	합산
Group A ($\alpha < 1$)	노이즈 제거 (이득)	균등화 (이득)	이중 이득
Group C ($\alpha \gg 1$)	다양성 손실 (손해)	집중화 (손해)	이중 손해

inner_sim 실험

Validation 누수 제거 후 Fold 01 결과 분석

`sim_inner_{method}.npy` (train_inner 전용) 적용

결과: `results/inner_sim/fold_01/` (2026-03-11)

inner_sim 실험: 무엇이 달라졌는가?

기존 파이프라인의 문제 (이슈 1 재확인)

`sim_*.npy`는 `train_full`(train_inner + validation, 90%)로 계산 → Phase 1 alpha 최적화 시 validation 데이터가 유사도에 이미 반영됨.

기존 방식

- 유사도: `train_full` 기반
- alpha 탐색 시 val 누수 발생
- Validation RMSE가 낙관적으로 측정됨

inner_sim 방식

- 유사도: **train_inner 전용**으로 재계산
- alpha 탐색 시 val 누수 없음
- 공정한 validation 평가 보장

검증 파일

Old	`results/fold_01/grid_search_results_reeval_lambda_0.002_v2.csv`
New	`results/inner_sim/fold_01/grid_search_results_20260311_132613.csv`

Alpha 최적값의 극적 변화 (K=20, TopN=20 기준)

Method	Old α	New α	Old Val RMSE	New Val RMSE	해석
chebyshev	7.50	1.00	0.804	1.011	극단 과적합 제거
msd	10.00	1.00	0.824	0.986	극단 과적합 제거
cpcc	7.00	1.00	0.811	0.997	극단 과적합 제거
euclidean	4.00	0.55	0.851	1.018	과적합 제거
pcc	3.50	0.55	0.826	1.016	과적합 제거
spcc	3.50	0.65	0.827	1.015	과적합 제거
kendall_tau_b	2.50	0.10	0.856	1.023	과적합 제거
manhattan	1.75	0.25	0.915	1.027	과적합 제거
acos	0.05	0.05	1.002	1.026	안정적 (변화 없음)
평균	**2.86**	**0.56**	**0.913**	**1.017**	

핵심

기존 방식에서 Val RMSE가 낙관적으로 측정될수록 alpha가 극단적으로 커졌음. inner_sim 적용 시 alpha가 **0.56**으로 수렴 (기존 **2.86**).

Validation–Test RMSE 갭: 과적합 정도 비교

설정	Val RMSE	Test RMSE	Val–Test 갭
Old (train_full 유사도)	0.913	1.026	+0.106 (심한 과적합)
New (inner_sim)	1.017	1.005	-0.010 (사실상 일치)

기존 방식 해석

- Val RMSE 0.913 → 실제보다 0.113 낮게 측정
- alpha 최적화가 실제보다 효과적으로 보임
- Test에서 +0.106 갭 → 전형적 data leakage 패턴

inner_sim 해석

- Val RMSE ≈ Test RMSE (갭: -0.01)
- 공정한 alpha 탐색 결과
- Test 성능이 Validation을 충실히 반영

Method별 Test RMSE: 공정 비교 포함 (전체 K/TopN 평균)

Method	Old (train_full)		New	Base	New 개선폭	
	λ=0.002	λ=0.030†	(inner, λ=0.002)	(α=1)	vs Old_{002}	vs Old_{030}
chebyshev	1.0923	1.0245	1.0015	1.0024	−0.091	−0.023
euclidean	1.0915	1.0413	1.0071	1.0077	−0.084	−0.034
manhattan	1.0657	1.0445	1.0153	1.0214	−0.050	−0.029
cpcc	1.0257	0.9809	0.9834	0.9836	−0.042	+0.003
pcc	1.0353	1.0088	1.0063	1.0075	−0.029	−0.003
spcc	1.0292	1.0064	1.0055	1.0061	−0.024	−0.001
kendall_tau_b	1.0321	1.0120	1.0098	1.0128	−0.022	−0.002
src	1.0309	1.0117	1.0095	1.0123	−0.021	−0.002
itr	1.0317	1.0272	1.0236	1.0271	−0.008	−0.004
ami	1.0241	1.0312	1.0260	1.0412	+0.002	−0.005
ari	1.0209	1.0275	1.0241	1.0434	+0.003	−0.003
acos	1.0126	1.0158	1.0114	1.0405	−0.001	−0.004
jmsd	0.9993	1.0019	1.0011	1.0029	+0.002	−0.001
msd	0.9695	0.9786	0.9760	0.9831	+0.007	−0.003
ipwr	0.9625	0.9641	0.9633	0.9646	+0.001	−0.001
cosine	1.0037	1.0051	1.0061	1.0068	+0.002	+0.001
jaccard	1.0096	1.0100	1.0110	1.0113	+0.001	+0.001
평균	1.0257	1.0113	**1.0048**	1.0103	**−0.021**	**−0.007**

† Old(λ=0.030)은 기존 방식에서의 최선 lambda. 개선: vs Old_{002} 는 9/17, vs Old_{030} 는 15/17.

Lambda별 Test RMSE: 대표값 선정 근거

설정	avg α	RMSE	대표
—— *Old* 방식 ——			
$\lambda = 0.001$	3.73	1.0269	
$\lambda = 0.002$	3.53	1.0257	●
$\lambda = 0.010$	2.26	1.0197	
$\lambda = 0.030$	1.56	1.0113	▲
—— *inner_sim* 방식 ——			
$\lambda = 0.001$	1.32	**1.0046**	★
$\lambda = 0.002$	1.16	1.0048	
$\lambda = 0.003$	0.96	1.0051	
$\lambda = 0.005$	0.79	1.0055	

대표값 선정 기준

- **동일 λ 비교** ($\lambda = 0.002$):
  누수 제거 효과 독립 측정.
  $\Delta = -0.021$ (과장 포함)
- **Old 최선 비교** ($\lambda = 0.030$):
  기존 최강 설정 대비.
  $\Delta = -0.007$ (공정한 수치)

선택

앞 표에 **두 열 모두 제시.**

단일 대표: **Old($\lambda = 0.030$) 대비** -0.007.

λ 패턴 역전

Old: $\lambda\uparrow \rightarrow$ alpha 억제 $\rightarrow$ RMSE$\downarrow$

inner: $\lambda\uparrow \rightarrow$ alpha 과억제 $\rightarrow$ RMSE$\uparrow$

inner_sim 실험 결론

핵심 발견

1. **Alpha 과적합 해소**: 평균 최적 α가 2.86 → 0.56으로 대폭 감소.
2. **Validation 신뢰도 회복**: Val–Test RMSE 갭이 +0.106 → -0.010으로 수렴.
3. **Test RMSE 개선**: 동일 $\lambda = 0.002$ 기준 -0.021; Old 최선($\lambda = 0.030$) 대비 -0.007.
4. **실제 최적화 이득**: inner_sim optimized가 baseline 대비 -0.005 ($\lambda = 0.001$ 최선).
5. **λ 패턴 역전**: Old는 $\lambda\uparrow$일수록 RMSE$\downarrow$ (누수 alpha 억제); inner_sim은 $\lambda\uparrow$일수록 RMSE$\uparrow$ (alpha 이미 수렴, 과억제 역효과).

가장 큰 개선 method

- chebyshev: -0.091 (과거 $\alpha = 7.5$ 문제 해결)
- euclidean: -0.084 (과거 $\alpha = 4.0$ 문제 해결)

전략적 시사점

이전 "baseline 우위" 결론은 data leakage에서 비롯된 것. inner_sim 기준에서 optimized가

V5 vs V6 전체 비교

Leakage 제거 효과: 10-Fold 완전 분석

V5: 유사도 `sim_{method}.npy` (full train 기반, validation leakage **있음**)

V6 inner_sim: 유사도 `sim_inner_{method}.npy` (train_inner 기반, leakage **없음**)

공통 조건: $\lambda = 0.002$, fold 1–10, 17개 유사도, $K = 10 \sim 100$, $N = 5 \sim 50$

분석 스크립트: `compare_v5_vs_v6_inner_sim.py`

핵심 결론: Leakage 제거 후 결론이 완전 역전

실험	Base RMSE	Opt RMSE	vs Base	Win Rate	Avg α
V5 (leakage 있음)	1.0102	1.0245	$+1.41\% \uparrow$	31.6%	2.761
V6 inner_sim (없음)	1.0134	**1.0083**	$\mathbf{-0.50\% \downarrow}$	**78.9%**	**1.096**

V5 → V6 절대값 변화

	Baseline	Optimized
V5	1.010179	1.024460
V6	1.013413	1.008300
Δ	$+0.003$	-0.016
변화율	$+0.32\%$	-1.58%

결론 역전

V5: Optimized $>$ Baseline (최적화 해로움)

V6: Optimized $<$ Baseline (**최적화 유익**)

이전 "$\alpha = 1$이 최강" 결론은

data leakage의 산물이었음.

Fold별 비교: 10개 Fold 전체 역전

V5 (leakage 있음): 전 fold 최적화 악화

Fold	Base	Opt	$\Delta\%$
1	1.0073	1.0197	$+1.24$
2	1.0058	1.0165	$+1.06$
3	1.0133	1.0254	$+1.19$
4	1.0135	1.0270	$+1.33$
5	0.9970	0.9973	$+0.03$
6	1.0098	1.0270	$+1.71$
7	1.0098	1.0296	$+1.96$
8	1.0274	1.0448	$+1.69$

V6 inner_sim: 전 fold 최적화 개선

Fold	Base	Opt	$\Delta\%$
1	1.0103	1.0048	-0.54
2	1.0085	1.0033	-0.52
3	1.0160	1.0113	-0.47
4	1.0166	1.0117	-0.48
5	1.0008	0.9954	-0.54
6	1.0139	1.0093	-0.46
7	1.0126	1.0078	-0.48
8	1.0312	1.0259	-0.52

유사도별 RMSE 상세 비교 (전체 fold × K × TopN 평균)

	V5 Base	V5 Opt		V6 Base	V6 Opt	
Method	RMSE	RMSE	$\Delta\%$	RMSE	RMSE	$\Delta\%$
ipwr	0.9706	0.9698	-0.08	0.9723	0.9717	-0.06
msd	0.9824	0.9776	-0.49	0.9871	0.9806	-0.66
cpcc	0.9835	1.0128	$+2.98$	0.9874	**0.9872**	-0.02
chebyshev	1.0044	1.0735	$+6.88$	1.0057	**1.0046**	-0.11
jmsd	1.0064	1.0052	-0.12	1.0074	1.0062	-0.13
spcc	1.0026	1.0224	$+1.97$	1.0080	**1.0076**	-0.04
pcc	1.0034	1.0261	$+2.27$	1.0093	**1.0085**	-0.08
euclidean	1.0086	1.0826	$+7.34$	1.0106	**1.0100**	-0.06
cosine	1.0098	1.0093	-0.06	1.0109	1.0106	-0.03
src	1.0089	1.0247	$+1.57$	1.0143	**1.0128**	-0.16
kendall_tau_b	1.0090	1.0266	$+1.75$	1.0148	**1.0129**	-0.18
jaccard	1.0144	1.0143	-0.01	1.0154	1.0153	-0.01
manhattan	1.0221	1.0669	$+4.38$	1.0235	**1.0181**	-0.53
itr	1.0298	1.0317	$+0.19$	1.0294	**1.0258**	-0.35
acos	1.0419	1.0152	-2.56	1.0411	1.0148	-2.52
ami	1.0378	1.0303	-0.72	1.0448	1.0284	-1.57
ari	1.0375	1.0268	-1.03	1.0461	1.0262	-1.90

굵음: V5 대비 크게 개선된 경우. $\Delta\% = (\text{Opt} - \text{Base})/\text{Base} \times 100$

유사도별 최적 α 비교: Leakage 전후

Method	V5 avg α	V6 avg α	$\Delta\alpha$	해석
chebyshev	7.60	1.43	-6.17	leakage로 극단값 선택됨
cpcc	6.26	1.05	-5.21	leakage로 극단값 선택됨
euclidean	4.39	0.89	-3.50	leakage로 극단값 선택됨
spcc	3.57	0.74	-2.83	leakage 해소 후 수렴
pcc	3.45	0.65	-2.80	leakage 해소 후 수렴
src	2.74	0.42	-2.31	leakage 해소 후 수렴
kendall_tau_b	2.66	0.41	-2.25	leakage 해소 후 수렴
itr	2.08	0.68	-1.40	leakage 해소 후 수렴
manhattan	2.05	0.44	-1.61	leakage 해소 후 수렴
jmsd	1.65	1.58	-0.06	영향 미미
cosine	1.41	1.21	-0.19	영향 미미
ipwr	1.24	1.20	-0.04	영향 없음
jaccard	1.06	1.07	$+0.01$	영향 없음
msd	5.30	6.00	$+0.70$	비선형 거동 유지
acos	0.27	0.75	$+0.49$	K=10 이상치 영향
ami	0.65	0.05	-0.60	누수 해소 후 수렴
ari	0.58	0.06	-0.52	누수 해소 후 수렴

chebyshev, euclidean, cpcc, pcc, spcc, src, kendall_tau_b, manhattan: V5에서 validation leakage로 인한 과도한 α 선택 → 테스트 성능 크게 악화.

K별 분해: Leakage 제거 후 K 의존성 변화

	V5		V6 inner_sim		V6 − V5	
K	Base	Opt ($\Delta\%$)	Base	Opt ($\Delta\%$)	Base 변화	Opt 변화
10	1.03521	1.03746 ($+0.22$)	1.03965	1.03066 (-0.86)	$+0.004$	-0.007
20	1.01369	1.02240 ($+0.86$)	1.01736	1.01171 (-0.56)	$+0.004$	-0.011
30	1.00813	1.02054 ($+1.23$)	1.01150	1.00672 (-0.47)	$+0.003$	-0.014
40	1.00630	1.02090 ($+1.45$)	1.00959	1.00505 (-0.45)	$+0.003$	-0.016
50	1.00577	1.02165 ($+1.58$)	1.00895	1.00450 (-0.44)	$+0.003$	-0.017
60	1.00580	1.02268 ($+1.68$)	1.00888	1.00441 (-0.44)	$+0.003$	-0.018
70	1.00611	1.02367 ($+1.75$)	1.00907	1.00459 (-0.44)	$+0.003$	-0.019
80	1.00649	1.02440 ($+1.78$)	1.00936	1.00481 (-0.45)	$+0.003$	-0.020
90	1.00693	1.02518 ($+1.81$)	1.00968	1.00512 (-0.45)	$+0.003$	-0.021
100	1.00735	1.02572 ($+1.82$)	1.01008	1.00545 (-0.46)	$+0.003$	-0.020

- **V5**: K 증가 → Optimized 악화 폭 확대 ($+0.22\% \rightarrow +1.82\%$)
- **V6**: K 증가 → Optimized 개선 수준 안정 ($-0.86\% \rightarrow -0.46\%$)
- Baseline은 두 실험 모두 K=50∼60 부근에서 최소 RMSE

MAD · Precision · Recall 비교 (all fold × K × TopN 평균)

MAD (낮을수록 좋음)

Method	V5 Opt	V6 Opt
ipwr	0.7718	0.7734
msd	0.7726	0.7756
cpcc	0.7887	**0.7779**
chebyshev	0.8383	**0.7982**
euclidean	0.8475	**0.8011**
manhattan	0.8406	**0.8080**
pcc	0.8101	**0.8015**
acos	0.8103	0.8099

Precision@N (높을수록 좋음)

Method	V5 Opt	V6 Opt
chebyshev	0.5942	**0.5972**
euclidean	0.5930	**0.5966**
manhattan	0.5929	**0.5962**
cpcc	0.5969	**0.5980**
acos	0.5979	0.5960
pcc	0.5974	**0.5982**

해석

- MAD에서도 leakage 문제가 있었던 method들(chebyshev, euclidean, cpcc, manhattan)은 V6에서 크게 개선
- Precision/Recall 변화는 RMSE/MAD에 비해 작음 ($\Delta < 0.005$)
- Precision/Recall은 최적화 방향이 두 실험에서 동일 (leakage 영향 제한적)

TopN 무관성 재확인

유사도 성능 순위 변화 (Optimized RMSE 기준)

Method	V5 순위	V6 순위	변화	원인
ipwr	1	1	—	안정
msd	2	2	—	안정
chebyshev	16	4	-12	α 과적합 해소
euclidean	17	8	-9	α 과적합 해소
cpcc	5	3	-2	α 과적합 해소
manhattan	15	14	-1	일부 해소
pcc	10	7	-3	α 수렴 후 개선
spcc	8	6	-2	α 수렴 후 개선
jmsd	3	5	$+2$	상대적 하락

Spearman $\rho = 0.534$

순위 상관이 중간 수준.

leakage 제거로 순위가 상당히 재편됨.

핵심 패턴

1. **크게 상승**: chebyshev, euclidean, cpcc, pcc, spcc → 이전 α 과대 추정이 문제
2. **크게 하락**: acos, jaccard, cosine → 상대적으로 leakage 수혜를 받았음
3. **안정 유지**: ipwr, msd → leakage 영향 없음

V5 vs V6 종합 결론

핵심 발견 5가지

1. **결론 역전**: leakage 제거 후 α 최적화가 전 fold에서 baseline을 개선 (-0.50%)
2. **Alpha 수렴**: 평균 최적 α가 2.76 → 1.10으로 정상화; chebyshev(7.6→1.4), cpcc(6.3→1.1) 등 극단값 해소
3. **Method별 이질성**: leakage 민감 그룹 (chebyshev, euclidean, cpcc, pcc 등)과 leakage 무관 그룹 (ipwr, msd, jmsd) 분리
4. **순위 재편**: Spearman $\rho = 0.534$ — 순위 상관이 중간 수준으로, method 선정 결론이 바뀜
5. **K 영향 역전**: V5에서 K 증가 시 최적화 효과 악화; V6에서 K 무관 안정 개선

이전 결론 (V5) 수정

- "$\alpha = 1$ baseline이 항상 최강"

실험 신뢰성 확보

V6 inner_sim은 validation 데이터 누수를

향후 실험 계획

실험 1: λ별 전체 비교 분석

- **목적**: 최적 λ 탐색 및 method별 λ 민감도 분석
- **범위**: $\lambda \in \{0.0001, 0.0002, \ldots, 0.030\}$, 전체 10-fold
- **분석 항목**
  - λ 증가에 따른 RMSE/MAD 커브 (method별)
  - 최적 λ vs baseline 크로스오버 지점
  - method별 λ 민감도 그룹핑
- **가설**: leakage 없는 환경에서는 λ가 작을수록 유리할 수 있음

실험 2: 음수 유사도 처리 방법 비교

- **목적**: 현재 clip to zero 방식 외 대안 검토
- **비교 방법**
  - **현행**: $\max(S, 0)$ (음수 제거)
  - **절댓값**: $|S|$ (부호 무시)
  - **유지**: 음수 그대로 사용 (반발 유저 정보 활용)
  - **Shift**: $S + |\min(S)|$ (최솟값 0 기준)
- **분석 항목**
  - method별 음수 비율 및 분포
  - 처리 방법별 RMSE/Precision 비교
  - 음수가 많은 method에서 효과 집중 분석

추가 분석

$\lambda = 0$ 조건에서의 과적합 양상 분석

및 LOO avg-α 중심 신규 페날티 비교

$\lambda = 0$ 조건: fold별 α 선택 분포 (과적합 신호)

Method별 α 통계 (10-fold 평균, 전체 K)

Method	Mean	Std	Min	Max
msd	9.995	0.050	9.5	10.0
acos	1.130	**1.901**	0.5	7.0
cosine	1.595	0.602	0.5	2.5
chebyshev	1.560	0.504	0.5	2.0
jmsd	1.955	0.477	0.5	2.5
jaccard	1.265	0.435	0.5	2.0
kendall	0.500	0.000	0.5	0.5
manhattan	0.500	0.000	0.5	0.5
src	0.500	0.000	0.5	0.5

과적합 신호 판정

- **msd**: $\alpha = 10$ (그리드 상한) 에 고착
  → $\lambda = 0$에서 극단값으로 치우침
- **acos**: std=1.901 로 **최대 분산**
  → K=10에서는 $\alpha = 7.0$, K≥20에서는 $\alpha = 0.5$ 선택
- K=50 단면: acos는 모든 fold에서 $\alpha = 0.5$ (std=0)
  → fold간 분산은 K=10이 주도

$\lambda = 0$ vs $\lambda = 0.002$ 평균 α 차이 (상위 4개)

Method	λ=0	λ=0.002	Δ

테스트 성능 비교: 4가지 조건 (10-fold 전체)

전체 평균 RMSE

조건	RMSE	vs Baseline
Baseline ($\alpha = 1$)	1.01341	—
현행 λ=0.002/α_0=1	**1.00829**	**−0.51%**
λ=0 (페날티 없음)	1.00890	−0.45%
λ=0.002/α_0=avg$_\alpha$ (신규)	1.00888	−0.45%

핵심 Method별 RMSE (전체 fold 평균)

Method	Base	현행	λ=0
acos	1.041	**1.015**	1.019
msd	0.987	0.981	**0.979**
cosine	1.011	1.011	**1.010**
jmsd	1.007	1.006	**1.006**

⇒ acos는 규제

유무에 따라 성능 차이가 명확 (-2.52% vs -2.11%)

Fold별 ΔRMSE (%, 현행 vs λ=0)

Fold	현행	λ=0	신규
1	−0.48%	−0.48%	−0.48%
2	−0.51%	−0.46%	−0.47%

신규 페날티 설계

$$\text{Score}(\alpha) = \text{MSE}_{\text{val}}(\alpha) + \lambda \cdot |\alpha - \overline{\alpha}_{\text{LOO}}|$$

추가 분석 결론: $\lambda = 0.002$ 현행 방식의 유효성 재확인

과적합 분석 결과

- **msd**: λ=0에서 $\alpha = 10$ 상한에 고착
  → 검증 데이터에 과적합. 하지만 테스트 RMSE는 λ=0이 오히려 우수 (-0.87% vs -0.66%) → 큰 α가 실제로도 유리한 방법
- **acos**: K=10에서 $\alpha = 7.0$ 선택 → fold간 분산 최대. 규제 적용 시 -2.52%, 무규제 시 -2.11%로 **규제 효과 뚜렷**
- 대부분의 method (13/17)는 fold간 α 분산이 낮음
  → 과적합 신호 약함

핵심 결론

1. **현행 λ=0.002가 최우수**: -0.51% (vs λ=0의 -0.45%)
   → 규제가 테스트 성능에 실질적으로 기여
2. **신규 페날티 무효**: avg$_\alpha$ 중심 페날티는 λ=0과 동등 (-0.45%)
   → 기존 방식 대비 개선 없음
3. **과적합은 제한적**: 대부분의 method는 λ=0에서도 안정적 α 선택
   → 규제 효과는 주로 acos · cosine에서 발생

종합

현행 λ=0.002/α_0=1 설정은 정당화됨. 신규 페날티 도입 필요 없음.

종합 결론

$\lambda = 0$ 실험 완료 — 15개 λ 조건 $\times$ 10-Fold 최종 비교

기존 추정치를 실제 실험 결과로 교체

λ별 Test RMSE 종합 비교 (V6 inner_sim, 10-Fold)

λ	Opt RMSE	vs Baseline	Win Rate	Avg α	비고
0	1.008089	$\mathbf{-0.525\%}$ ↓	**88.6%**	1.354	**최대 개선**
0.0005	1.008112	-0.523% ↓	87.5%	1.312	
0.001	1.007825	-0.523% ↓	85.0%	1.258	**최저 절대 RMSE**
0.002	1.007980	-0.508% ↓	78.8%	1.102	현행 설정
0.003	1.008248	-0.482% ↓	71.6%	0.917	
0.005	1.008594	-0.447% ↓	55.8%	0.788	
0.010	1.009011	-0.406% ↓	35.9%	0.791	
0.030	1.009625	-0.346% ↓	23.9%	0.875	
Baseline ($\alpha = 1$)		RMSE $\approx$ 1.0134 (기준선)			

기존 추정 대비 수정

이전 분석에서는 $\lambda = 0$: -0.45%로 추정했으나, **실제 실험 결과** -0.525%로 현행(-0.508%)을 상회. λ 증가 시 성능이 **단조 감소**하는 패턴 확인.

Fold별 ΔRMSE: $\lambda = 0$ vs 0.001 vs 0.002 ($-\% =$ 개선)

	$\lambda = 0$		$\lambda = 0.001$		$\lambda = 0.002$	
Fold	Opt	$\Delta\%$	Opt	$\Delta\%$	Opt	$\Delta\%$
1	1.0045	**−0.567**	1.0046	−0.559	1.0048	−0.544
2	1.0031	**−0.536**	1.0032	−0.532	1.0033	−0.518
3	1.0112	**−0.479**	1.0112	−0.476	1.0113	−0.466
4	1.0116	**−0.496**	1.0116	−0.493	1.0117	−0.479
5	0.9953	**−0.555**	0.9953	−0.554	0.9954	−0.541
6	1.0091	**−0.473**	1.0092	−0.471	1.0093	−0.460
7	1.0075	**−0.506**	1.0076	−0.499	1.0078	−0.482
8	1.0256	**−0.543**	1.0257	−0.535	1.0259	−0.517
9	1.0038	**−0.571**	1.0039	−0.563	1.0041	−0.542
10	1.0093	**−0.522**	1.0093	−0.515	1.0095	−0.497
AVG	**1.0081**	**−0.525**	1.0082	−0.520	1.0083	−0.505

핵심 관찰

$\lambda = 0$이 **10개 fold 전부**에서 $\lambda = 0.002$를 상회. $\lambda = 0 \approx 0.001 > 0.002$ 순서가 모든 fold에서 일관됨.

보조 지표 비교: MAD · Precision · Recall (전체 fold×method×K×TopN)

	MAD		Precision		Recall	
λ	Opt	$\Delta\%$	Opt	$\Delta\%$	Opt	$\Delta\%$
0	0.80077	-0.466%	0.59735	$+0.017\%$	0.95029	$+0.010\%$
0.001	0.80086	-0.466%	0.59783	$+0.018\%$	0.95019	$+0.011\%$
0.002	0.80099	-0.450%	0.59783	$+0.018\%$	0.95019	$+0.011\%$
Baseline	0.80452	—	0.59725	—	0.95019	—

- MAD도 RMSE와 동일 패턴: $\lambda = 0$이 가장 큰 개선 (-0.466%)
- Precision · Recall은 모든 λ에서 baseline과 거의 동일 ($< 0.02\%$)
- $\Rightarrow \alpha$ 최적화는 **RMSE/MAD 개선**에 효과적이며, Precision/Recall에는 중립

Method별 비교: $\lambda = 0$, Opt vs Baseline ($\alpha = 1$) (10-fold 평균)

Method	Base	Opt	$\Delta\%$	WinR%
acos	1.0411	1.0148	−2.526	100
ari	1.0461	1.0262	−1.897	100
ami	1.0448	1.0285	−1.562	100
msd	0.9871	0.9785	−0.871	100
manhattan	1.0234	1.0180	−0.532	100
itr	1.0294	1.0255	−0.375	94
kendall_tau_b	1.0148	1.0129	−0.181	96
jmsd	1.0074	1.0058	−0.158	90
src	1.0143	1.0128	−0.154	94

Method	Base	Opt	$\Delta\%$	WinR%
chebyshev	1.0056	1.0044	−0.121	88
pcc	1.0093	1.0084	−0.090	99
ipwr	0.9723	0.9716	−0.073	78
cosine	1.0109	1.0102	−0.064	70
euclidean	1.0106	1.0100	−0.061	71
spcc	1.0080	1.0075	−0.051	86
cpcc	0.9874	0.9872	−0.022	80
jaccard	1.0154	1.0152	−0.018	60
전체	**1.0134**	**1.0081**	**−0.525**	**88.6**

핵심

K별 비교: $\lambda = 0$, Opt vs Baseline (10-fold × 17 method × all TopN 평균)

K	Base	Opt	$\Delta\%$	WinR%
10	1.0396	1.0305	-0.880	85.3
20	1.0174	1.0114	-0.582	84.1
30	1.0115	1.0064	-0.504	84.1
40	1.0096	1.0047	-0.481	88.8
50	1.0089	1.0042	-0.469	90.0
60	1.0089	1.0042	-0.465	90.6
70	1.0091	1.0044	-0.462	91.2
80	1.0094	1.0047	-0.461	91.8

K 패턴 해석

- K=10에서 개선폭 최대 (-0.880%), K 증가할수록 감소
- K≥50 이후 $\Delta\%$가 $\approx -0.46\%$로 수렴
- Win Rate는 K=10에서 가장 낮고 K=80 근방에서 최고
- K=10의 큰 개선: 소수 이웃에서 α 효과가 더 크게 발현

TopN 효과

TopN(5–50) 변화가 RMSE에 **영향 없음**: 모든

$\lambda = 0$ Alpha 선택: Method $\times$ K 평균 (10-fold $\times$ all TopN 평균)

Method	K=10	K=20	K=30	K=40	K=50	K=60	K=70	K=80	K=90	K=100	Mean	Std	Min	Max
msd	9.950	10.000	10.000	10.000	10.000	10.000	10.000	10.000	10.000	10.000	9.995	0.050	9.5	10.0
acos	**6.800**	0.050	0.050	0.050	0.055	0.060	0.070	0.080	0.100	0.105	0.742	2.022	0.05	7.0
jmsd	1.110	1.400	1.665	1.850	1.990	2.110	2.215	2.275	2.355	2.405	1.938	0.467	0.70	2.75
cosine	0.715	0.935	1.190	1.385	1.660	1.755	1.845	2.050	2.115	2.200	1.585	0.576	0.25	2.50
chebyshev	0.375	0.905	1.265	1.500	1.665	1.785	1.880	1.955	2.015	2.070	1.542	0.525	0.30	2.15
ipwr	0.690	0.915	1.120	1.250	1.345	1.420	1.470	1.520	1.565	1.585	1.288	0.299	0.55	1.75
jaccard	0.710	0.855	1.025	1.165	1.240	1.360	1.415	1.515	1.560	1.595	1.244	0.393	0.25	2.00
cpcc	0.410	0.745	0.925	1.070	1.160	1.250	1.310	1.375	1.425	1.470	1.114	0.344	0.10	1.70
spcc	0.260	0.405	0.475	0.540	0.620	0.675	0.730	0.755	0.795	0.825	0.608	0.225	0.05	1.00
pcc	0.200	0.345	0.420	0.480	0.545	0.605	0.645	0.680	0.725	0.755	0.540	0.211	0.05	0.90
euclidean	0.215	0.460	0.665	0.825	0.925	1.005	1.080	1.135	1.180	1.220	0.871	0.320	0.10	1.30
itr	**0.000**	0.010	0.050	0.185	0.410	0.460	0.535	0.635	0.785	0.940	0.401	0.352	0.00	1.35
manhattan	0.110	0.215	0.315	0.370	0.425	0.450	0.495	0.530	0.550	0.565	0.402	0.149	0.05	0.60
src	0.100	0.175	0.235	0.285	0.335	0.385	0.415	0.440	0.480	0.500	0.335	0.154	0.05	0.60
kendall_tau_b	0.090	0.165	0.220	0.290	0.320	0.370	0.405	0.435	0.470	0.485	0.325	0.150	0.05	0.55
ari	0.050	0.050	0.050	0.050	0.055	0.055	0.055	0.050	0.055	0.055	0.052	0.026	0.00	0.10
ami	0.050	0.050	0.050	0.050	0.050	0.045	0.045	0.040	0.025	0.025	0.043	0.021	0.00	0.10

$\lambda = 0$ Alpha 선택: Method × Fold (all K × TopN 평균)

Method	F1	F2	F3	F4	F5	F6	F7	F8	F9	F1
msd	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	9.95	10.
acos	0.75	0.74	0.72	0.76	0.79	0.76	0.71	0.70	0.76	0.7
jmsd	2.20	1.95	1.56	1.73	2.18	2.02	1.79	2.19	1.99	1.7
cosine	1.90	1.83	1.13	1.28	1.84	1.54	1.62	1.80	1.77	1.1
chebyshev	1.55	1.58	1.53	1.64	1.48	1.48	1.45	1.50	1.61	1.6
ipwr	1.38	1.48	1.19	1.30	1.32	1.30	1.31	1.23	1.24	1.1
jaccard	1.57	1.23	0.74	0.97	1.57	1.25	1.11	1.50	1.35	1.1
cpcc	1.18	1.10	1.06	1.40	0.99	1.20	1.13	1.03	0.95	1.1
euclidean	0.87	0.88	0.90	0.98	0.81	0.86	0.83	0.81	0.90	0.9
itr	0.38	0.49	0.54	0.38	0.35	0.41	0.24	0.28	0.36	0.5
pcc	0.64	0.58	0.47	0.61	0.42	0.69	0.36	0.60	0.37	0.6
spcc	0.73	0.63	0.52	0.68	0.47	0.79	0.43	0.68	0.42	0.7
manhattan	0.40	0.39	0.44	0.48	0.37	0.40	0.38	0.35	0.42	0.4
src	0.21	0.38	0.30	0.41	0.30	0.40	0.32	0.43	0.20	0.4
kendall_tau_b	0.23	0.36	0.29	0.39	0.29	0.41	0.30	0.41	0.19	0.4
ari	0.04	0.07	0.03	0.03	0.07	0.05	0.08	0.05	0.05	0.0

주요 패턴

그리드 상한 고착:

- msd: 전 fold $\alpha = 10.0$

K 의존성 강함:

- acos: K=10→α=6.8, K≥20→≈0.05 (이분)
- itr: K=10→0, K=100→0.94 (단조 증가)
- jmsd/cosine/chebyshev: K↑⇒ α ↑

Fold간 안정:

- chebyshev, euclidean, manhattan: std<0.2

λ 역전 메커니즘 및 논문 최종 권고

V5 vs V6의 λ 효과 대비

	V5 (leakage)	V6 (inner_sim)
최적 λ	0.030	**0**
Avg α	2.76	**1.10**
$\lambda\uparrow$ 효과	개선	**악화**

메커니즘

- V5: leakage로 α가 과대 추정 ($\gg 1$)
  $\Rightarrow \lambda|\alpha - 1|$이 극단값 억제 → 유익
- V6: $\alpha \approx 1.1$로 자연 수렴
  $\Rightarrow$ 규제가 미세한 추가 억제 → 해로움

최종 결론 (3가지)

1. **α 최적화는 유효**: 모든 λ에서 baseline 대비 개선 (10-fold 전체)
2. **정규화 불필요**: inner_sim에서 α가 자연 수렴하므로, $\lambda > 0$은 순수 손실
3. **이전 결론 수정**: "λ=0.002 최우수"는 추정 기반이었으며, 실험에서 역전

논문 권고

- 최종 보고: $\lambda = 0$ 또는 $\lambda \leq 0.001$
- Win Rate: 88.6% (vs 78.8% at λ=0.002)
- ΔRMSE: -0.525% (vs -0.508%)

2026-04-15 업데이트

similarities.py 구현 버그 발견

영향 범위: msd / jmsd / acos — results/inner_sim/ 전용

발견된 버그 요약

핵심

`utils/similarities.py` 의 **msd** 와 **acos** 구현에 수식 오류가 있었음.

`jmsd` 는 `msd()` 를 내부 호출하므로 버그를 자동 상속.

메서드	버그 내용	실제 영향
`msd`	$\frac{x}{5} - \frac{y}{5}$ 사용 (rating min $r_{\min} = 1$ 무시) (정상: $\frac{x-y}{4}$)	값 범위 $[0.36,\ 1.0]$ 으로 압축 (정상: $[0,\ 1]$)
`jmsd`	`msd()` 호출로 버그 상속	msd 와 동일
acos	user-mean center-	값 범위 $[-0.013,\ +0.059]$ 으로

버그 상세: msd 와 acos 수식 비교

msd 수식 비교

버그 (현재)

$$\text{MSD}(u,v) = 1 - \frac{\sum_{i \in I_{uv}} \left(\frac{x_i}{5} - \frac{y_i}{5}\right)^2}{|I_{uv}|}$$

최대 제곱항 $= (1-0)^2/25 = 0.04$ → 분모가 과소

수정 후

$$\text{MSD}(u,v) = 1 - \frac{\sum_{i \in I_{uv}} \left(\frac{x_i - y_i}{4}\right)^2}{|I_{uv}|}$$

최대 제곱항 $= 4^2/16 = 1$ → 정상 범위 $[0,1]$

acos 수식 비교

버그 (현재)

user-mean centering: $\bar{r}_u = \frac{1}{|I_u|} \sum_i r_{ui}$

$$\text{ACOS}(u,v) = \frac{\sum_i (x_i - \bar{r}_u)(y_i - \bar{r}_v)}{\sqrt{\cdots}\sqrt{\cdots}}$$

`nan_to_num` 로 결측 처리 → 평균 0으로 치환

수정 후

item-mean centering: $\bar{r}_i = \frac{1}{|\mathcal{U}_i|} \sum_u r_{ui}$

$$\text{ACOS}(u,v) = \frac{\sum_{i \in I_{uv}} (x_i - \bar{r}_i)(y_i - \bar{r}_i)}{\sqrt{\cdots}\sqrt{\cdots}}$$

공통 평점 셀만 사용, 결측 제외

영향 범위: results/inner_sim/ 만 해당

재실험 범위 한정

이후 실험은 **inner_sim 파이프라인**만 사용한다.

results/fold_*/ (과거 V5 실험) 와 results/*.csv (최상위 집계)는 내부 참조용으로만 보존.

경로	해당 여부	사유
results/inner_sim/fold_01∼10/	해당	버기 msd/jmsd/acos 행 포함
results/inner_sim/combined/	해당	fold 전체 집계 포함
results/fold_*/	해당 없음	과거 V5, 논문에서 비교용만
results/*.csv	해당 없음	최상위 집계, 참조용만

파일 규모 (inner_sim 기준)

- fold_01: 26 CSV (≈3400 rows/파일)

.npy 파일 영향

- fold별 sim_inner_msd.npy

버그가 기존 결과에 미친 영향

msd: $\alpha = 10$ 상한 고착 현상 재해석

이전 분석에서 $\lambda = 0$ 조건의 msd가 **모든 fold에서** $\alpha = 10.0$ (그리드 상한)에 고착됨.

버그로 인해 유사도 범위가 $[0.36,\ 1.0]$으로 압축 → alpha grid에서 항상 상한이 최적으로 선택될 가능성. 수정 후 α 패턴이 달라질 것으로 예상.

acos: K=10 이상치 ($\alpha = 6.8$) 재해석

이전 분석에서 acos의 K=10 단면만 $\alpha \approx 6.8$, K≥20에서는 $\alpha \approx 0.05$로 이분화.

버그로 인해 아코사인 유사도 범위가 $[-0.013,\ +0.059]$으로 압축 → 거의 모든 유사도가 ≈ 0이어서 K=10에서 최적 α가 불안정하게 결정될 가능성.

수정 후 기대 변화

- msd: α 분포 정상화 (10.0 고착 해소)
- acos: K=10 이상치 해소, 전체 α 범위 안정화

재실험 5단계 계획

Phase 0 — 아카이빙 + 행 삭제

1. `results/inner_sim/` → `results/archive_buggy_20260415/` 복사
2. 원본 CSV에서 msd/jmsd/acos 행 삭제 (14개 정상 메서드 행만 보존)
3. `sim_inner_{msd,jmsd,acos}.npy` → `*_buggy.npy` 이름 변경

Phase 1 — 코드 수정

- `similarities.py`: msd $\div 4$, acos item-mean
- `main_experiment_v6_inner_sim.py`:

Phase 2 — 검증

`similarity_verification.ipynb` 에서 수정된 구현 assert 확인:

- msd U1↔U4 ≈ 0.375
- acos 값 범위 $\in [-1, +1]$

Phase 3 — .npy 재생성

`precompute_similarity_inner.py` 실행 (3개 메서드만, 14개는 기존 파일 재사용)

Phase 4 — 실험 재수행

`main_experiment_v6_inner_sim.py` 실행

재실험 조건: $\lambda = 0$ 채택 근거

이전 분석 결론 (2026-04-08 업데이트 기준)

λ	Opt RMSE	vs Baseline	Win Rate
0	**1.008089**	$\mathbf{-0.525\%}$	**88.6%**
0.001	1.007825	-0.523%	85.0%
0.002	1.007980	-0.508%	78.8%

$\lambda = 0$ 채택 이유

- inner_sim 환경에서 α가 자연 수렴 ($\bar{\alpha} \approx 1.1$)
- $\lambda > 0$은 이미 수렴한 α를 추가 억제 → 순손실

주의

msd/jmsd/acos 버그 수정 후

14개 정상 메서드의 수치는 변하지 않는다.

Appendix: 사용한 핵심 수식

$$S_{uv}^{(\alpha)} = \max(S_{uv}, 0)^{\alpha}$$

$$\hat{r}_{ui} = \frac{\sum_{v \in \mathcal{N}_K(u,i)} S_{uv}^{(\alpha)} r_{vi}}{\sum_{v \in \mathcal{N}_K(u,i)} S_{uv}^{(\alpha)}}$$

$$\mathsf{Score}(\alpha) = \mathsf{MSE}_{val}(\alpha) + \lambda|\alpha - 1|$$

Appendix: 결과 해석 시 주의점

- 문서/CSV에 historical run과 corrected run이 공존함
- 본 발표는 주요 결론($\alpha = 1$ 우위, regularization 한계)에 초점을 둠
- 숫자 표기는 각 파일의 공식 집계값을 직접 인용함